MODULE 26 · EXAM STRATEGY - ALL BLOCKS

Practice paper 2

Thirty new questions, none repeating paper 1's topics, with three deliberate pairs that change one detail from a paper 1 question.

QUESTIONS 30 TIMING about 80 s each MODULE 26 of 28

admin@sthithapragna.com

Same rules as paper 1

- **No paper.** Trace in your head.
- Commit to an answer before each walkthrough. Pause if you need more time.
- Tally your misses by block, and compare them with paper 1.
- If paper 1 showed a weak block, check whether it has improved.

The user types 3.5 and presses Enter.

What is the output of this program?

```
1  x = input()
2  print(float(x) * 2, x * 2)
```

A `7.0 7.0`

B `7.0 3.53.5`

C `7 3.53.5`

D The code raises a TypeError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = input()
2  print(float(x) * 2, x * 2)
```

A `7.0 7.0`
*x is a str; x * 2 repeats it.*

B `7.0 3.53.5`
float(x) * 2 is 7.0; the str x repeated is 3.53.5.

C `7 3.53.5`
float() gives a float, so 7.0.

D The code raises a TypeError
*float * int and str * int are both legal.*

BLOCK 1

input() returns a str. Only the converted copy is a number.

What is the output of this program?

```
count = 5
while count < 3:
    count += 1
else:
    count *= 2
print(count)
```

A 5

B 6

C 10

D 12

◆ PAUSE AND DECIDE

What is the output of this program?

```
count = 5
while count < 3:
    count += 1
else:
    count *= 2
print(count)
```

A 5

The loop's else runs even after zero passes.

B 6

5 < 3 is False at the first test: no passes.

C 10

Zero passes, no break, so the else doubles count.

D 12

The body never runs.

BLOCK 2

A loop's else runs whenever no break fired, including after zero passes.

What is the output of this program?

```
1  a = [1, 2, 3, 4, 5]
2  a[1:3] = []
3  del a[-1]
4  print(a)
```

A [1, 4, 5]

B [1, 2, 3, 4]

C [4, 5]

D [1, 4]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [1, 2, 3, 4, 5]
2  a[1:3] = []
3  del a[-1]
4  print(a)
```

A `[1, 4, 5]`
del a[-1] removes the 5.

B `[1, 2, 3, 4]`
a[1:3] = [] removes 2 and 3.

C `[4, 5]`
The 1 at index 0 is not in the slice.

D `[1, 4]`
Remove indexes 1 and 2, then the last element.

BLOCK 3

Assigning [] to a slice deletes it.

What is the output of this program?

```
1  def add(item, box=[]):
2      box.append(item)
3      return box
4
5  a = add(1)
6  b = add(2)
7  print(a, b, a is b)
```

A [1] [2] False

B [1] [1, 2] False

C [1, 2] [1, 2] True

D [1, 2] [2] True

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def add(item, box=[]):
2      box.append(item)
3      return box
4
5  a = add(1)
6  b = add(2)
7  print(a, b, a is b)
```

A `[1] [2] False`
That was paper 1's answer, with a None default.

B `[1] [1, 2] False`
a and b are the same list.

C `[1, 2] [1, 2] True`
One default list, shared by both calls.

D `[1, 2] [2] True`
Both names refer to the one list.

BLOCK 4

Paired with paper 1, question 26: a default list is created once and shared.

What is the output of this program?

```
1  print(0 or "" or "x", 3 > 2 > 1)
```

A True True

B 0 True

C x False

D x True

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(0 or "" or "x", 3 > 2 > 1)
```

A `True True`
or returns an operand, not a bool.

B `0 True`
0 is falsy, so or moves on.

C `x False`
3 > 2 > 1 means 3 > 2 and 2 > 1: True.

D `x True`
or returns the first truthy operand; the chain is True.

BLOCK 1

or returns an operand. A chain is an and of neighbouring pairs.

What is the output of this program?

```
score = 85
if score > 50:
    grade = "C"
elif score > 80:
    grade = "B"
else:
    grade = "F"
print(grade)
```

A C

B B

C F

D C, then B

◆ PAUSE AND DECIDE

What is the output of this program?

```
score = 85
if score > 50:
    grade = "C"
elif score > 80:
    grade = "B"
else:
    grade = "F"
print(grade)
```

A C

85 > 50 is checked first and is True.

B B

The more specific test comes second, too late.

C F

else only runs if both tests fail.

D C, then B

Only one branch of a chain runs.

BLOCK 2

The first true branch wins. Put the most demanding test first.

What is the output of this program?

```
1  a = [1, 2, 3]
2  b = a
3  b += [4]
4  a[0] = 9
5  print(a, b)
```

A `[9, 2, 3] [1, 2, 3, 4]`

B `[9, 2, 3, 4] [9, 2, 3, 4]`

C `[9, 2, 3] [9, 2, 3, 4]`

D `[1, 2, 3, 4] [9, 2, 3, 4]`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [1, 2, 3]
2  b = a
3  b += [4]
4  a[0] = 9
5  print(a, b)
```

A `[9, 2, 3] [1, 2, 3, 4]`
That was paper 1's answer, with b = b + [4].

B `[9, 2, 3, 4] [9, 2, 3, 4]`
+= extends the shared list; then a[0] = 9 shows in both.

C `[9, 2, 3] [9, 2, 3, 4]`
a and b are still one list.

D `[1, 2, 3, 4] [9, 2, 3, 4]`
a and b are one list, so they print the same.

BLOCK 3

Paired with paper 1, question 28: += changes the shared list; b = b + ... builds a new one.

What is the output of this program?

```
try:
    x = 1 / 0
except ZeroDivisionError:
    print("E", end="")
else:
    print("L", end="")
finally:
    print("F", end="")
```

A ELF

B LF

C E

D EF

◆ PAUSE AND DECIDE

What is the output of this program?

```
try:
    x = 1 / 0
except ZeroDivisionError:
    print("E", end="")
else:
    print("L", end="")
finally:
    print("F", end="")
```

A ELF

else runs only when there was no exception.

B LF

There was an exception, so except runs, not else.

C E

finally always runs.

D EF

Exception: except, then finally.

BLOCK 4

Exception handled: except, then finally. No exception: else, then finally.

What is the output of this program?

```
1  print("cat" < "Cat", "2" > "10")
```

A True False

B True True

C False True

D False False

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print("cat" < "Cat", "2" > "10")
```

A `True False`
Lower case letters come after capitals.

B `True True`
c (99) is after C (67).

C **`False True`**
c is greater than C; "2" beats "1" at the first character.

D `False False`
"2" > "10" compares "2" with "1" first.

BLOCK 1

Strings compare by code point, character by character: digits, capitals, lower case.

What is the output of this program?

```
lines = 0
for i in range(3):
    for j in range(3):
        if j > i:
            break
        lines += 1
print(lines)
```

A 9

B 3

C 1

D 6

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  lines = 0
2  for i in range(3):
3      for j in range(3):
4          if j > i:
5              break
6          lines += 1
7  print(lines)
```

A 9

break cuts each inner loop short.

B 3

break ends the inner loop only, not the outer.

C 1

The outer loop still makes three passes.

D 6

Inner passes: 1, then 2, then 3.

BLOCK 2

break ends the innermost loop only.

What is the output of this program?

```
d = {"x": 1}
for k in ["x", "y", "x"]:
    d[k] = d.get(k, 0) + 1
print(d)
```

A `{'x': 2, 'y': 1}`

B `{'x': 1, 'y': 1}`

C `{'x': 3, 'y': 1}`

D The code raises a KeyError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  d = {"x": 1}
2  for k in ["x", "y", "x"]:
3      d[k] = d.get(k, 0) + 1
4  print(d)
```

A `{'x': 2, 'y': 1}`
x is counted twice in the loop, plus its starting 1.

B `{'x': 1, 'y': 1}`
Each pass adds 1 to the stored count.

C **`{'x': 3, 'y': 1}`**
x: 1 + 1 + 1. y: get gives 0, plus 1.

D The code raises a KeyError
get supplies 0 for the missing y.

BLOCK 3

get(k, 0) + 1 counts safely, even for new keys.

What is the output of this program?

```
1  def check(n):
2      if n < 0:
3          raise ValueError("negative")
4      return n ** 0.5
5
6  try:
7      print(check(-4))
8  except ValueError as e:
9      print("error:", e)
```

A `2.0`

B `error: negative`

C The program stops with an unhandled ValueError

D `error: ValueError`

◆ PAUSE AND DECIDE

What is the output of this program?

```
def check(n):
    if n < 0:
        raise ValueError("negative")
    return n ** 0.5

try:
    print(check(-4))
except ValueError as e:
    print("error:", e)
```

A `2.0`
-4 fails the test, so the function raises first.

B `error: negative`
raise ValueError with a message; the handler prints it.

C The program stops with an unhandled ValueError
The except branch matches ValueError.

D `error: ValueError`
e prints the message, not the class name.

BLOCK 4

raise sends the exception to the caller's handler. Printing e shows its message.

What is the output of this program?

```
1  print(round(4.5), round(5.5), int(-4.9))
```

A 4 6 -4

B 5 6 -5

C 4 5 -5

D 5 6 -4

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(round(4.5), round(5.5), int(-4.9))
```

A `4 6 -4`

Halves go to the even neighbour; int() cuts towards zero.

B `5 6 -5`

round sends 4.5 to 4, and int(-4.9) is -4.

C `4 5 -5`

5.5 rounds to the even 6.

D `5 6 -4`

4.5 is an exact half. it goes to 4.

BLOCK 1

round: halves to even. int(): towards zero.

What is the output of this program?

```
for i in range(10, 0, -4):
    print(i, end=" ")
```

A 10 6 2 -2

B 10 6 2

C 6 2

D 10 6

◆ PAUSE AND DECIDE

What is the output of this program?

```
for i in range(10, 0, -4):
    print(i, end=" ")
```

A `10 6 2 -2`

-2 is past the stop of 0.

B `10 6 2`

Start included; keep stepping by -4 while above 0.

C `6 2`

The start, 10, is always the first value.

D `10 6`

2 is still above the stop.

BLOCK 2

Start is included; stop is never reached, in either direction.

What is the output of this program?

```
1  s = "banana"
2  s.replace("a", "o")
3  t = s.upper()
4  print(s, t[1:4])
```

A banana ANA

B bonono ONO

C bonono ANA

D banana BAN

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  s = "banana"
2  s.replace("a", "o")
3  t = s.upper()
4  print(s, t[1:4])
```

A `banana ANA`

replace's result is thrown away; t is the upper-case copy.

B `bonono ONO`

s.replace on its own line changes nothing.

C `bonono ANA`

s is unchanged, so it still prints banana.

D `banana BAN`

t[1:4] is indexes 1, 2 and 3.

BLOCK 3

A string method's result must be assigned, or it is lost.

What is the output of this program?

```
1  def show(n):
2      if n > 0:
3          print(n, end="")
4          show(n - 1)
5          print(n, end="")
6
7  show(2)
```

A 21

B 1221

C 2112

D 2121

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def show(n):
2      if n > 0:
3          print(n, end="")
4          show(n - 1)
5          print(n, end="")
6
7  show(2)
```

A 21

The print after the call also runs, on the way back.

B 1221

The first print happens before going deeper.

C 2112

Down: 2, 1. Back up: 1, 2.

D 2121

The second prints run in reverse order, deepest first.

BLOCK 4

Work before the recursive call runs going down; work after it runs coming back up.

What is the output of this program?

```
1  print(1.5e3, 2e-5)
```

A 1500 0.00002

B 1500.0 2e-05

C 1.5e3 2e-5

D 1500.0 0.00002

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(1.5e3, 2e-5)
```

A `1500 0.00002`
An E makes a float, and tiny floats print in E notation.

B `1500.0 2e-05`
1.5e3 is the float 1500.0; 2e-5 prints in E notation.

C `1.5e3 2e-5`
Python prints the value, not the way it was written.

D `1500.0 0.00002`
Below 1e-4, Python switches to E notation.

BLOCK 1

E always means float. Very small and very large floats print in E notation.

What happens when this program runs?

```
1  i = 10
2  while i != 0:
3      i -= 3
4  print(i)
```

A 0 is printed

B -2 is printed

C 1 is printed

D Nothing is printed: the loop never ends

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  i = 10
2  while i != 0:
3      i -= 3
4  print(i)
```

A 0 is printed

i never equals 0.

B -2 is printed

The loop does not stop at -2: -2 != 0.

C 1 is printed

1 != 0, so the loop continues.

D Nothing is printed: the loop never ends

10, 7, 4, 1, -2 … skips 0 for ever.

BLOCK 2

With != as the test, check the stop value can be hit exactly.

What is the output of this program?

```
1  a, b, c = (1, 2, 3)
2  a, c = c, a
3  print(a, b, c)
```

A 1 2 3

B 3 2 3

C 1 2 1

D 3 2 1

◆ PAUSE AND DECIDE

What is the output of this program?

```
a, b, c = (1, 2, 3)
a, c = c, a
print(a, b, c)
```

A 1 2 3

The second line swaps a and c.

B 3 2 3

Both values are read before either name changes.

C 1 2 1

The swap is simultaneous.

D **3 2 1**

Unpack 1, 2, 3, then swap a and c.

BLOCK 3

The right side is packed first, then unpacked: a true swap.

What is the output of this program?

```
x = 5
def f(x):
    x += 1
    return x

print(f(x), x)
```

A 6 5

B 6 6

C 5 5

D The code raises an UnboundLocalError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 5
2  def f(x):
3      x += 1
4      return x
5
6  print(f(x), x)
```

A `6 5`

The parameter x is local; the global x is untouched.

B `6 6`

x += 1 changes only the parameter.

C `5 5`

f returns the incremented local x.

D `The code raises an UnboundLocalError`

x is a parameter, so it has a value from the start.

BLOCK 4

A parameter is a local variable, even when it shares a global's name.

What is the output of this program?

```
1  x = 3
2  x *= x + 1
3  x -= 2
4  print(x)
```

A 7

B 8

C 12

D 10

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 3
2  x *= x + 1
3  x -= 2
4  print(x)
```

A 7

*x *= x + 1 means x = x * (x + 1).*

B 8

*Reads line 2 as x * x + 1, then misses the - 2.*

C 12

Misses the final x -= 2.

D 10

3 * (3 + 1) is 12, then 12 - 2.

BLOCK 1

x op= expr means x = x op (expr).

What is the output of this program?

```
for i in range(5):
    if i == 2:
        continue
    if i == 4:
        break
    print(i, end="")
```

A 01234

B 0134

C 013

D 01

◆ PAUSE AND DECIDE

What is the output of this program?

```
for i in range(5):
    if i == 2:
        continue
    if i == 4:
        break
    print(i, end="")
```

A `01234`
Ignores both continue and break.

B `0134`
break stops the loop before 4 prints.

C **`013`**
0, 1; 2 skipped by continue; 3; break at 4.

D `01`
continue skips only 2; the loop carries on.

BLOCK 2

continue skips one pass; break ends the loop.

What is the output of this program?

```
m = [[1, 2], [3, 4], [5, 6]]
print(m[2][0] + m[-2][1], len(m[0]))
```

A 8 2

B 9 2

C 9 3

D The code raises an IndexError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  m = [[1, 2], [3, 4], [5, 6]]
2  print(m[2][0] + m[-2][1], len(m[0]))
```

A `8 2`
m[-2] is [3, 4], so m[-2][1] is 4.

B `9 2`
m[2][0] is 5; m[-2][1] is 4; each row has 2.

C `9 3`
len(m[0]) counts one row's elements: 2.

D `The code raises an IndexError`
m has rows 0, 1 and 2.

BLOCK 3

Row first, then column. len(m) counts rows; len(m[0]) counts one row.

What is the output of this program?

```
def span(a):
    return min(a), max(a)

lo, hi = span([3, 8, 1])
print(hi - lo, type(span([1])))
```

A `7 <class 'list'>`

B `1 <class 'tuple'>`

C `The code raises a TypeError`

D `7 <class 'tuple'>`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def span(a):
2      return min(a), max(a)
3
4  lo, hi = span([3, 8, 1])
5  print(hi - lo, type(span([1])))
```

A `7 <class 'list'>`

return a, b packs a tuple, not a list.

B `1 <class 'tuple'>`

hi - lo is 8 - 1.

C `The code raises a TypeError`

A function may return a tuple, and it unpacks into two names.

D `7 <class 'tuple'>`

min 1, max 8; a comma-separated return is one tuple.

BLOCK 4

return a, b returns one tuple, which can be unpacked.

> Each literal is typed on its own.

Which two are valid integer literals? (Choose two.)

A `1_000`

B `0o78`

C `0x1F`

D `1__0`

◆ PAUSE AND DECIDE

Which two are valid integer literals? (Choose two.)

A `1_000`

One underscore between digits is fine.

B `0o78`

8 is not an octal digit. SyntaxError.

C `0x1F`

Hex digits include A to F: 31.

D `1__0`

Two underscores in a row are not allowed.

BLOCK 1

Each base allows only its own digits. Underscores: one at a time, between digits.

What is the output of this program?

```
n = 1
while n < 50:
    n *= 2
    if n == 27:
        break
else:
    print("done")
print(n)
```

A 64

B done, then 64

C 27

D done, then 27

◆ PAUSE AND DECIDE

What is the output of this program?

```
n = 1
while n < 50:
    n *= 2
    if n == 27:
        break
else:
    print("done")
print(n)
```

A 64

No break fired, so the else runs first.

B done, then 64

n doubles past 27 without hitting it: no break, so else runs.

C 27

Doubling from 1 never gives 27.

D done, then 27

n is never 27.

BLOCK 2

Paired with paper 1, question 10: whether break fires decides whether the else runs.

> Each line is run on its own.

Which two lines print None? (Choose two.)

A `print([1].append(2))`

B `print([3, 1].pop())`

C `print(sorted([2, 1]))`

D `print([2, 1].reverse())`

◆ PAUSE AND DECIDE

Which two lines print None? (Choose two.)

A `print([1].append(2))`
append changes the list and returns None.

B `print([3, 1].pop())`
pop returns the removed element: 1.

C `print(sorted([2, 1]))`
sorted returns a new list: [1, 2].

D `print([2, 1].reverse())`
reverse changes the list in place and returns None.

BLOCK 3

Methods that change a list return None. pop is the exception: it returns the element.

What is the output of this program?

```
1  def count(n):
2      while n > 0:
3          yield n
4          n -= 1
5
6  g = count(3)
7  print(next(g), sum(g))
```

A 3 6

B 3 3

C 3 0

D [3] 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def count(n):
2      while n > 0:
3          yield n
4          n -= 1
5
6  g = count(3)
7  print(next(g), sum(g))
```

A `3 6`

next already took the 3.

B `3 3`

next takes 3; sum adds what is left: 2 + 1.

C `3 0`

The generator still holds 2 and 1.

D `[3] 3`

next gives a value, not a list.

BLOCK 4

A generator remembers its place. Each value can be taken only once.

The user types 4, 9, 3, then -1, pressing Enter after each.

What is the output of this program?

```
1  best = int(input())
2  n = best
3  while n >= 0:
4      if n > best:
5          best = n
6      n = int(input())
7  print(best)
```

A 9

B 4

C -1

D 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  best = int(input())
2  n = best
3  while n >= 0:
4      if n > best:
5          best = n
6      n = int(input())
7  print(best)
```

A 9

best keeps the largest value seen; -1 stops the loop.

B 4

9 is larger, so best is replaced.

C -1

The sentinel stops the loop and is never compared.

D 3

best keeps the largest, not the last.

BLOCK 2

The sentinel is read and tested, never processed.

> Consider try statements in Python in general.

Which two statements about except branches are true? (Choose two.)

A. A bare except: must be the last branch

B. Two except branches can run for one exception

C. except (A, B): handles either exception

D. finally runs only when no exception occurs

◆ PAUSE AND DECIDE

Which two statements about except branches are true? (Choose two.)

A. A bare except: must be the last branch
Anywhere else, it is a SyntaxError.

B. Two except branches can run for one exception
At most one branch runs.

C. except (A, B): handles either exception
A tuple names several exceptions.

D. finally runs only when no exception occurs
finally always runs.

BLOCK 4

One branch at most; bare except last; finally always.

How did you do?

SCORE OUT OF 30	WHAT IT MEANS	WHAT TO DO NEXT
27 to 30	comfortably above the pass mark	take paper 3
21 to 26	at or above 70%, the pass mark	compare blocks with paper 1, revisit the weakest, then paper 3
Below 21	below the pass mark	rework the weakest block before paper 3

ON THE EXAM

Compare with paper 1. A score that rose after revisiting one block shows the method works.

Find the block, then the modules

BLOCK	QUESTIONS IN THIS PAPER	MODULES TO REVISIT
1: Fundamentals	1, 5, 9, 13, 17, 21, 25	1 to 5
2: Control flow	2, 6, 10, 14, 18, 22, 26, 29	6 to 10
3: Data collections	3, 7, 11, 15, 19, 23, 27	11 to 15
4: Functions and exceptions	4, 8, 12, 16, 20, 24, 28, 30	16 to 21

WHAT MATTERS MOST

Misses that cluster in the same block across two papers are a real weakness. Fix that block first.